

MDT Implementation

Repository: MrunalJCrazy/Duranta

Fork of: OpenAirInterface (OAI) NR RRC

Feature: MDT — Minimization of Drive Tests

1. What is MDT, in plain English?

Mobile networks need to know where coverage is weak. Traditionally, engineers did this by physically driving around a city with test phones, recording signal strength as they went — literally a "drive test." It works, but it's slow and expensive.

MDT (Minimization of Drive Tests) is a 3GPP feature that lets ordinary phones already on the network do this job automatically in the background. The phone quietly notices things like "my signal just got weak here" and reports that back to the network, so engineers don't have to send a car out.

This repository adds a first, simplified version of that idea into OAI's 5G (NR) RRC code — the part of the software that manages the connection between a phone ("UE") and a tower ("gNB").

Key terms used throughout this document

- **UE (User Equipment)** — the phone / modem, i.e. the client side of the connection.
- **gNB (next-generation Node B)** — the 5G base station / tower, i.e. the network side.
- **RRC (Radio Resource Control)** — the signalling protocol UE and gNB use to set up and manage the radio connection.
- **RSRP** — Reference Signal Received Power, a number in dBm that says how strong the signal is (closer to 0 is stronger, e.g. -80 dBm is good, -110 dBm is poor).
- **ASN.1** — the formal language 3GPP uses to define RRC messages; OAI has generated C code from it that this project reuses.

2. The big picture: how the pieces fit together

The feature has two halves that mirror each other, because a phone call always has two sides:

- **UE side (the phone):** Watches the signal strength, decides when something worth reporting has happened, keeps a small in-memory log, and eventually packages that log into a standard RRC "MeasurementReport" message to send to the tower.
- **gNB side (the tower):** Receives that same MeasurementReport message (it doesn't need a new message type — MDT reuses the existing measurement-report pipeline), pulls the interesting numbers back out of it, and stores them per-UE in memory.

Seven files were touched to build this, split evenly between the two sides plus one shared file that both sides rely on:

- UE side: rrc_defs.h, rrc_UE.c

- Shared message-building code: `asn1_msg.h`, `asn1_msg.c`
- gNB side: `nr_rrc_defs.h`, `rrc_gNB.c`, `rrc_gNB_UE_context.c`

The sections below go through each file one at a time, explaining first what the file is normally responsible for in OAI, and then what was specifically added for MDT.

openair2/RRC/NR_UE/rrc_defs.h

What this file normally does

This is a header file — it doesn't run any logic itself, it just defines the "shapes" of data (structs, enums, constants) that the UE-side RRC code uses everywhere else. Think of it as the blueprint file: `rrc_UE.c` can't compile without it.

What was added for MDT

A small, self-contained set of MDT building blocks was added so the UE can keep track of its own logging state:

- **NR_MDT_LOG_BUFFER_SIZE:** A constant (64) that fixes how many measurement samples the UE will remember before it must send a report. It's a simple ring buffer size, not something read from the network.
- **nr_mdt_trigger_reason_t:** An enum — basically a labelled list of "why did we log this sample?" options: logging was just turned on, RSRP dropped suddenly, RSRP is simply low, a routine measurement update happened, or it's a periodic check.
- **nr_mdt_sample_t:** A struct describing one logged measurement: which radio frame it happened in, which cell/gNB it came from, whether it was a neighbor cell, the RSRP value, how much it changed since last time, and the trigger reason.
- **nr_mdt_state_t:** A struct that holds the UE's whole MDT "state machine": whether MDT is enabled, whether logging is currently active, the thresholds used to decide when to trigger, counters, and the fixed-size array of `nr_mdt_sample_t` entries.
- **mdt field:** A single `nr_mdt_state_t` field named `mdt` was added into the existing per-neighbor-gNB struct, so every connection the UE has gets its own independent MDT tracker.

In short: this file adds the "memory shape" for MDT on the phone side — no logic, just data definitions that `rrc_UE.c` fills in.

openair2/RRC/NR_UE/rrc_UE.c

What this file normally does

This is the main UE-side RRC engine — the biggest and busiest file in the UE stack. It processes incoming radio messages, tracks connection state, and reacts to events like handovers or measurement reports. Existing code already handles the normal MeasurementReport flow (the standard mechanism a phone uses to tell the tower "here's my signal strength"); MDT deliberately reuses that same machinery instead of inventing a new one.

What was added for MDT

A handful of small helper functions were added, plus a few hooks into existing logic:

- **nr_mdt_reason_to_string():** Turns the trigger-reason enum into a readable string, purely for log messages.
- **nr_rrc_mdt_reset():** Clears out a connection's MDT state and sets the default thresholds — a 3 dB drop counts as significant, and -100 dBm counts as "low" signal.

- **nr_rrc_mdt_enable():** Turns MDT logging on for a given connection to a gNB, resetting state the first time it's enabled and printing a log line confirming the thresholds in use.
- **nr_rrc_mdt_store_sample():** Writes one new sample into the ring buffer described above, wrapping around once the buffer is full, and logs a one-line summary of each sample as it's stored.
- **nr_rrc_mdt_flush_report():** Takes everything currently sitting in the ring buffer and, for each sample, calls into `asn1_msg.c` to actually build and send an RRC MeasurementReport message. Afterwards it resets the buffer counters.

These helpers are then wired into the UE's existing measurement-handling code: whenever a normal measurement update comes in, the UE now also compares the new RSRP against the previous one, decides whether the drop is big enough or the signal is low enough to count as an MDT trigger, and — if logging is active — stores a sample and/or flushes a report. Enabling is likewise hooked into the point where the UE would already process the network's request to enable it.

openair2/RRC/NR/MESSAGES/asn1_msg.h

What this file normally does

This header declares the functions that build actual over-the-air RRC messages, encoded in the official 3GPP ASN.1 format. It's shared: both UE code and gNB code can include it, though in this project it's the UE side that calls into it to build reports.

What was added for MDT

One new function declaration: `do_nrMDTMeasurementReport_SA()`. It takes the same kind of inputs as a normal measurement report (which measurement ID triggered it, what quantity was measured, the serving cell ID, the RSRP index, and optionally a neighbor cell's ID and RSRP) and declares that it will fill a buffer with the properly encoded message, ready to transmit.

openair2/RRC/NR/MESSAGES/asn1_msg.c

What this file normally does

This is where the functions declared in `asn1_msg.h` are actually implemented — the code that assembles a C struct representing an RRC message and then runs the ASN.1 encoder over it to produce the raw bytes that go over the air.

What was added for MDT

The new `do_nrMDTMeasurementReport_SA()` function was added, but deliberately as a thin wrapper: it simply forwards all of its arguments to the existing `do_nrMeasurementReport_SA()` function that OAI already had for ordinary measurement reports.

This is an important design choice worth calling out: rather than inventing a brand-new MDT-specific message format, the implementation reuses the standard MeasurementReport message as-is. That keeps things simple and compatible with the existing gNB decoder, at the cost of not yet carrying any MDT-specific fields (like timestamps or trigger reasons) over the air — those currently only exist in the UE's local log.

openair2/RRC/NR/nr_rrc_defs.h

What this file normally does

This is the gNB-side counterpart to `rrc_defs.h` — it defines the data structures the tower's RRC code uses, including the big per-UE context struct that holds everything the network currently knows about one connected phone.

What was added for MDT

- **nr_mdt_report_t:** A struct representing one MDT report as stored on the network side: when it arrived, which UE and measurement ID it belongs to, the serving cell's ID and RSRP, and — if present — a neighbor cell's ID and RSRP plus a flag saying whether neighbor data was actually included.
- **mdt_reports and mdt_report_count:** Two new fields added to the UE context struct: a growable list (`seq_arr_t`) that holds every `nr_mdt_report_t` received from that UE, and a simple counter of how many have arrived so far.

Compare this with `nr_mdt_sample_t` on the UE side: the UE's version is about raw, frequent measurement samples kept in a small fixed ring buffer; the gNB's version is about decoded reports, kept in a growable list, one entry per MeasurementReport message actually received over the air.

openair2/RRC/NR/rrc_gNB.c

What this file normally does

This is the main gNB-side RRC engine, the tower's equivalent of `rrc_UE.c`. Among many other things, it already contains the logic that receives a MeasurementReport message from a UE, decodes it, and reacts to it (for example, to decide on a handover).

What was added for MDT

- **store_mdt_report():** A new function that takes a decoded MeasurementReport and pulls out the fields relevant to MDT: it reads the serving cell's ID and RSRP (checking whether the value came from an SSB or CSI-RS reference signal), and, if present, does the same for the first neighbor cell in the list. It stamps the report with the current time and the UE's ID, appends it to that UE's `mdt_reports` list, increments the counter, and logs a one-line summary.

This function is called right after the existing measurement-report handling code finishes processing an incoming report — so MDT storage happens as a side effect of the normal measurement pipeline, without changing how that pipeline behaves for anyone not using MDT.

Note the RSRP conversion used here: the raw index from the message is converted to a dBm value by adding 157, which is the standard 3GPP mapping formula for this particular RSRP index range.

openair2/RRC/NR/rrc_gNB_UE_context.c

What this file normally does

This file manages the lifecycle of a UE's context on the gNB — creating a fresh context struct when a phone first connects, and cleaning it up (freeing memory) when the phone disconnects.

What was added for MDT

- **On context creation:** `seq_arr_init()` is called to properly initialize the new `mdt_reports` list, so it's ready to have reports pushed onto it as soon as they arrive.

- **On context teardown:** `seq_arr_free()` is called to release the memory used by that UE's `mdt_reports` list, preventing a memory leak when the phone disconnects.

This is a small but necessary piece of plumbing: without it, the growable list added in `nr_rrc_defs.h` would either start in an unsafe, uninitialized state or leak memory over the lifetime of the network.

3. Known limitations (as of this version)

This is intentionally described as functional scaffolding rather than a standards-complete MDT feature. A few gaps are worth knowing about:

- **Simplified payload:** the over-the-air message is a normal `MeasurementReport`, so MDT-specific details (timestamps, trigger reason, buffer index) never leave the UE — the gNB only sees what a standard report already carries.
- **Incomplete 3GPP semantics:** real MDT has configuration messages, logging modes, and area-scope options defined by the standard; this implementation uses fixed, hard-coded thresholds and a simple always-on style of logging instead.
- **In-memory-only storage on the gNB:** reports live in the `mdt_reports` list only as long as the process is running and the UE stays connected; nothing is written to disk or a database.
- **No export pipeline:** there is currently no mechanism to pull the collected reports out of the gNB for offline analysis (e.g. a file dump, an API, or a database sink) — the data is logged to the console and kept in memory, but not exported anywhere yet.

4. Quick glossary

RRC	Radio Resource Control — the signalling protocol between phone and tower.
UE	User Equipment — the phone/modem.
gNB	The 5G base station (tower).
RSRP	Reference Signal Received Power — a measure of signal strength, in dBm.
SSB / CSI-RS	Two different types of reference signals a phone can measure RSRP against.
ASN.1	The formal notation 3GPP uses to define the exact byte-level format of RRC messages.
MeasurementReport	The standard RRC message a UE sends to report signal measurements; MDT reuses it as-is.
Ring buffer	A fixed-size array that wraps around and overwrites its oldest entries once full.
MDT	Minimization of Drive Tests — using ordinary phones to gather coverage data instead of manual drive tests.